Revision of VLDB Submission #1058

We thank the reviewers for their careful assessment. This response states our conclusions, presents the new measurements, and quotes the proposed manuscript wording at its intended insertion point. The quoted passages are included here so that each change can be assessed without consulting the paper. Section and figure numbers refer to the submitted manuscript; the submitted PDF itself has not been edited.

Experimental scope used throughout this response

The measured training runs use one server with eight 32 GiB V100 GPUs, a local batch of 8,192 unless noted, preshuffled inputs resident in host memory, and a 510-step GPU training window that includes online mapping and communication. RAPID targets row-wise embedding sharding, so the TorchRec comparison uses row-wise sharding. HugeCTR, TorchRec, EMBark, Herald, BagPipe, and RAPID use AdaGrad 0.01. HET-GMP uses its native shared-table optimizer, SGD 0.01, except for the marked Kaggle/DCN run. The submitted paper's original hardware was eight 16 GiB PCIe V100s; results on the present 32 GiB machine are new measurements, not identical-hardware replications. The throughput window excludes offline planning, input preparation, initialization, and validation.

Proposed manuscript text — Section 7.1, “Hardware and Training Configuration,” after the existing hardware paragraph:

> In the revision experiments, we additionally use one server with eight 32 GiB V100 GPUs. Each rank processes 8,192 samples per step unless otherwise stated. Inputs are shuffled before training and preloaded in memory. Reported throughput measures 510 GPU training steps, including online ID mapping, scheduling, and communication, but excluding one-time preprocessing, initialization, and validation. RAPID optimizes row-wise embedding placement; accordingly, the TorchRec comparison uses a row-wise sharding plan. HET-GMP uses its native shared-table SGD update at learning rate 0.01, whereas the other systems use AdaGrad at learning rate 0.01, subject to the exception identified in the results table.

Meta Revision 1 — External baselines, workloads, and training results

Response. We report measured eight-GPU training comparisons with TorchRec, EMBark, HET-GMP, Herald, and BagPipe. RAPID has the highest measured throughput in all 12 listed dataset/model combinations. The three main rows retain the submitted paper's model shapes: Kaggle/DCN uses 26 tables, 214 sparse features, dimension 32, a six-layer cross network and two 1,024-wide deep layers; Terabyte/DLRM-DCNv2 uses 26 tables, 214 sparse features, dimension 32, three cross layers, a 512-dimensional projection, and a `[1024,1024,512,256,1]` top MLP; Taobao/DeepFM uses four tables, four sparse features, dimension 128, three 400-wide deep layers, and first- and second-order FM terms. The other nine rows are dimension-32 model extensions.

Table R1. Measured eight-GPU training throughput (samples/s).

Dataset / model	HugeCTR	TorchRec	EMBark	HET-GMP	Herald	BagPipe	RAPID
Kaggle / DLRM	1,265,110	568,823	1,804,285	629,040	10,459	73,470	2,136,522
Kaggle / DCN, dim32	625,151	560,969	1,701,025	701,298†	11,319	73,040	2,032,420
Kaggle / Wide&Deep	1,251,291	533,137	1,796,028	775,392	9,848	70,144	2,292,587
Kaggle / DeepFM	1,405,858	556,847	2,111,551	816,947	9,845	71,503‡	2,855,600
Terabyte / DLRM	1,273,833	568,210	1,849,886	653,264	10,459	99,594	2,010,277
Terabyte / DLRM-DCNv2, dim32	546,769	489,409	1,258,722	666,211	12,562	96,550	1,329,655
Terabyte / Wide&Deep	1,262,214	532,223	1,878,571	814,181	10,344	94,800	2,107,001

Dataset / model	HugeCTR	TorchRec	EMBark	HET-GMP	Herald	BagPipe	RAPID
Terabyte / DeepFM	1,417,189	556,198	2,236,009	867,333	10,377	96,978‡	2,577,394
Taobao / DLRM	4,588,460	2,147,182	4,027,885	3,230,496	415,450	2,064,385	4,947,326
Taobao / DLRM-DCNv2	3,306,323	1,802,251	3,002,414	2,379,513	424,153	1,981,837	3,488,199
Taobao / Wide&Deep	6,472,360	2,393,622	5,305,386	5,532,203	434,392	2,096,891	7,255,623
Taobao / DeepFM, dim128	5,561,281	1,006,574	3,443,948	5,470,898	362,719	1,332,057‡	8,338,178

† HET-GMP diverged at SGD 0.01 in this case; this entry uses a smaller initialization and SGD 0.001. ‡ The indicated DeepFM runs had unstable or high final loss and are throughput observations only, not accuracy comparisons. HugeCTR uses all Reduction on the Kaggle and Terabyte main rows and all Unique on the Taobao main row; RAPID uses CPS and per-table ACP. The nine extension rows were obtained from the completed dimension-32 runs. The new Herald main-model run uses cache threshold 100 and eight LAIA threads per rank. The different native training engines and HET-GMP optimizer mean that Table R1 is a systems comparison, not a controlled kernel-only comparison.

The three larger-table workloads T180, T200, and T510 come from the HugeCTR synthetic-data generator, which samples IDs with Zipf exponent 1.1 using each table's vocabulary and hotness. The full official configurations do not fit as end-to-end training on this eight-GPU machine at their original per-table dimensions: the idealized FP32 embedding weights alone require approximately 30.9, 48.4, and 53.4 GiB per GPU, respectively, before optimizer states and training buffers. T180's generator actually emits 177 tables. We therefore report communication only on one 65,536-sample shard of each workload, preserving table count and hotness while using dimension 32 and local batch 512. These are 80 global batches over five passes, not full official throughput runs.

Table R2. Eight-rank forward-plus-backward remote embedding payload on T-series shards (GiB).

Workload	HugeCTR	TorchRec	EMBark	HET-GMP	Herald	BagPipe	RAPID
T180	73.311	96.797	96.797	157.642	241.929	763.835	64.752
T200	76.259	109.375	109.375	52.306	84.617	229.810	37.835
T510	64.700	278.906	278.906	41.318	71.119	226.686	24.024

On the corresponding dimension-32 Criteo inputs, RAPID/HET-GMP forward-plus-backward remote payloads are 15.066/15.829 GiB for Kaggle and 10.785/11.553 GiB for Terabyte, over eight ranks and five passes. RAPID reduces these by 4.82% and 6.65%. HET-GMP transfers less on Taobao. RAPID and HET-GMP both exploit access relationships, but RAPID retains the original global-batch membership, combines CPS with per-table ACP, enforces equal main-embedding capacity, and places newly observed IDs without moving existing rows. HET-GMP uses a different Hetu/HetuTable execution engine, so the throughput gap is not attributable to transfer volume alone.

Proposed manuscript text — Section 7.1, replace “Baselines” and append to “Models and Datasets”; Section 7.2, after the original end-to-end result:

> We extend the evaluation from HugeCTR to TorchRec with row-wise sharding, EMBark, HET-GMP, Herald, and BagPipe. Table R1 reports measured eight-GPU training throughput on Kaggle, Terabyte, and Taobao across DLRM, DCN, Wide&Deep, and DeepFM. The paper's main model shapes are Kaggle/DCN (26 tables, 214 sparse features, dimension 32, six cross layers and two 1,024-wide deep layers), Terabyte/DLRM-DCNv2 (26 tables, 214 sparse features, dimension 32, three cross layers, 512 projection, and `[1024,1024,512,256,1]` top MLP), and Taobao/DeepFM (four tables, dimension 128, three 400-wide deep layers with first- and second-order FM terms). The remaining nine configurations use dimension 32. RAPID has the highest measured throughput in all 12 combinations. HET-GMP uses its native SGD shared-table update, and several indicated baseline runs do not provide a stable-loss accuracy comparison.

> For table-rich workloads, we generate T180, T200, and T510 from the HugeCTR generator with Zipf(1.1) IDs and the specified table vocabularies and hotness. The official full-size configurations exceed the available memory for end-to-end training at their original dimensions. We instead compare remote embedding payloads on a common 65,536-sample shard, preserving the table structure but using dimension 32 and local batch 512. RAPID transfers 64.752, 37.835, and 24.024 GiB, respectively, over five passes; these are lower than all six measured alternatives on the same shards. We do not interpret this shard-level communication result as full-configuration training throughput.

Proposed manuscript text — Section 8, add after “Embedding Placement, Caching, and Replication”:

> HET-GMP is a particularly close predecessor to CPS: it builds sample–embedding and embedding co-occurrence graphs and jointly partitions samples and embeddings to reduce communication. RAPID differs in preserving the membership of every original global batch, applying per-table ACP after CPS, enforcing an equal-capacity main embedding placement, and supporting placement of new IDs without moving existing rows. Herald and BagPipe use caching and scheduling paths with different execution costs. We compare all of these systems directly on measured training throughput, and compare remote embedding payload on the larger T-series shards; because their training engines differ, throughput differences cannot be attributed solely to the placement algorithm.

Meta Revision 2 — Limits of physical scalability

Response. We agree that the submitted 16–128-rank results are trace-driven transfer and load counts, not measured multi-node runtime. The only physical end-to-end speedups demonstrated here are on one eight-GPU server. We will restrict the abstract, Section 7.6, and conclusion accordingly; multi-node throughput remains unmeasured.

Proposed manuscript text — Abstract, replace the final scalability sentence; Section 7.6, replace the opening paragraph; Section 9, replace the scalability claim:

> Abstract: On a single eight-GPU server, RAPID improves measured training throughput. Trace-driven experiments with 16–128 logical ranks report transfer-volume and load trends; multi-node training speedup has not been measured.
>
> Section 7.6: The 16–128-rank experiments replay placement and sample routing on recorded accesses. They count remote embedding vectors and rank loads, but do not execute distributed training at those scales. In particular, the replay does not measure network topology, contention, synchronization, or overlap. We therefore interpret these results as communication-volume scalability, while all runtime and speedup claims are limited to the measured eight-GPU server.
>
> Section 9: RAPID's measured training gains are established on one eight-GPU server. Larger logical-rank studies support the transfer-volume trend but leave physical multi-node scalability for future evaluation.

Meta Revision 3 — Offline plans and incremental updates

Response. We evaluated whether a plan can be extended as data are appended. The initial plan uses 2,097,152 preshuffled Criteo day-6 multi-hot samples. We then add 56 nonoverlapping windows of 65,536 samples and compare four strategies on the same final eight windows (W49–W56). RAPID assigns only new IDs and schedules newly added samples; owner and physical ID of every existing row remain unchanged, so the measured migration count of existing rows is zero. Incremental updates reduce forward remote receives by 73.96% against hash-placed all Unique and by 29.88% against retaining the old plan with round-robin placement for new IDs. Full replanning has lower traffic, at the cost of changing the existing plan. This same-day append experiment does not establish robustness to real temporal distribution shift or quantify the time to recompute and distribute the plan.

Table R3. Mean forward remote vectors received per rank per step on W49–W56.

Strategy	Placement of new IDs	Vectors	Reduction vs. hash all Unique
Hash all Unique	Hash placement	480,344.75	—
Keep old plan	Round robin	178,368.19	62.87%
RAPID incremental	Co-access-aware	125,080.08	73.96%
Full replanning	Rebuild the global plan	103,956.46	78.36%

The 480,344.75 figure is one-way remote receive per rank per step. A forward-plus-backward count of the same payload would be approximately twice that number; these metrics should not be mixed. Table R3 compares one common sample sequence. It measures payload and existing-row migration, not planning wall time.

Proposed manuscript text — Section 7.8, replace “Sustainability under Incremental Data Updates”:

> We test incremental adaptation on a preshuffled Criteo day-6 multi-hot stream. An initial plan is built from 2,097,152 samples; 56 disjoint 65,536-sample windows are then appended. We compare hash-placed all Unique, retention of the old plan with round-robin placement for new IDs, RAPID's co-access-aware incremental placement, and full replanning on the same final eight windows. Their mean forward remote receives are 480,344.75, 178,368.19, 125,080.08, and 103,956.46 vectors per rank per step. RAPID's incremental path lowers traffic by 73.96% relative to hash all Unique and by 29.88% relative to retaining the old plan. It does not change the owner or physical ID of any previously placed row, so existing-row migration is zero. These windows test data growth from the same shuffled day, not a real temporal distribution shift, and the comparison does not include plan recomputation or distribution time.

Meta Revision 4 — Batch size, peak load, and planning parameters

Response. Batch size affects deduplication. On the same 262,144 Criteo samples and eight logical ranks, increasing local batch from 512 to 16,384 reduces RAPID's forward remote receives per sample from 20.485 to 5.965; its reduction relative to all Unique increases from 77.30% to 88.36%. Each all-Unique baseline is recalculated at the same batch size. This is a communication sensitivity result; batch-dependent throughput and gradient-accumulation behavior have not been measured in this experiment.

Table R4. Batch-size sensitivity: forward remote vectors per input sample.

Local batch	All Unique	RAPID	RAPID reduction
512	90.250	20.485	77.30%
1,024	82.145	17.070	79.22%
2,048	74.045	13.170	82.21%
4,096	66.145	10.380	84.31%
8,192	58.540	7.960	86.40%
16,384	51.230	5.965	88.36%

The busiest rank receives at most 1.197 times the per-rank mean on the three measured real datasets. Main embedding storage is constrained to equal row counts, differing by at most one row per rank. Optimizer state and communication buffers are not part of this storage measure.

Table R5. Peak forward receive load and main-embedding capacity, eight ranks.

Dataset	Mean vectors/rank/step	Busiest rank/step	Receive peak/mean	Main rows/rank, min–max	Storage peak/mean
Kaggle	98,739.62	103,057.56	1.044	2,409,915–2,409,916	1.000000207
Terabyte	70,678.40	84,585.56	1.197	2,682,414–2,682,415	1.000000326

Dataset	Mean vectors/rank/step	Busiest rank/step	Receive peak/mean	Main rows/rank, min–max	Storage peak/mean
Taobao	5,829.11	6,003.38	1.030	120,622–120,623	1.000002073

For the parameter study, the defaults are β=0.0001, γ=0.2, and hot-ID Top-K=40,000. One factor changes at a time. “Reverse profiling order” reverses only the order of profiling batches used to construct the plan; it leaves sample membership and the training batches unchanged. The measured main-embedding storage peak/mean remains 1.000000 to six decimals.

Table R6. One-factor changes in forward remote receives per sample.

Plan setting	Vectors/sample	Change from default	Main-storage peak/mean
Default	7.962	—	1.000000
γ=0	11.635	+46.1%	1.000000
β=0	8.097	+1.7%	1.000000
Hot Top-K=100,000	8.938	+12.3%	1.000000
Reverse profiling-batch order	7.976	+0.2%	1.000000

Proposed manuscript text — Section 7.8, add after “Load Balance”; Section 5.2, add parameter values after the CPS algorithm:

> We vary local batch size on the same 262,144 Criteo samples with eight ranks and recompute the all-Unique reference for each batch size. From local batch 512 to 16,384, RAPID's forward remote receives fall from 20.485 to 5.965 vectors per sample, while the reduction relative to all Unique rises from 77.30% to 88.36%. Larger batches offer more within-batch duplicate IDs for deduplication. This experiment measures transfer counts, not batch-specific training throughput or gradient accumulation.
>
> Across Kaggle, Terabyte, and Taobao, the busiest rank receives 1.044, 1.197, and 1.030 times the average forward remote vectors per step, respectively. Main embedding storage is capacity constrained: the maximum and minimum rank allocations differ by at most one row, yielding peak-to-mean ratios of 1.000000207, 1.000000326, and 1.000002073. These ratios exclude optimizer states and temporary buffers.
>
> We set β=0.0001, γ=0.2, and hot-ID Top-K=40,000 by default. Relative to the default 7.962 forward remote vectors per sample, setting γ=0 gives 11.635, β=0 gives 8.097, Top-K=100,000 gives 8.938, and reversing the profiling-batch order gives 7.976. The capacity constraint keeps the main embedding row count balanced in every case. Reversing profiling order does not change the training samples or global batches.

Meta Revision 5 — Cold-ID co-occurrence and accuracy

Response. We address the concern that a high maximum co-occurrence ratio (MCR) may come from IDs seen only once or twice. In a 2,097,152-sample Criteo multi-hot workload, an ID is cold if it appears in fewer than 0.01% of samples (fewer than 210). Support is the number of distinct samples containing the ID; repeated occurrences within one sample count once. For each target cold ID, MCR is the largest co-occurrence count with any other cold ID divided by the target's support. Setting a minimum support of 100 therefore retains only target IDs seen 100–209 times. The proportion with MCR>95% remains 85.50% at this threshold. This is a workload-specific statistic: the multi-hot expansion can create deterministic co-occurrence, so it is not evidence that the same rate holds for unexpanded categorical data or future time windows.

Table R7. Cold IDs with MCR>95% in the evaluated multi-hot workload.

Minimum target support	Eligible cold IDs	IDs with MCR>95%	Share
10	1,886,103	1,333,054	70.68%
20	953,848	770,892	80.82%
50	353,257	299,515	84.79%
100	129,222	110,490	85.50%

RAPID preserves the samples belonging to each original global batch and only changes their rank assignment. This preserves the batch objective and aggregate gradient under rank-independent sample computation and identical optimizer updates, up to floating-point reduction order. It does not, by itself, establish bitwise identity for rank-local state or stochastic operations. In a full one-epoch Taobao/DLRM run (23,920,640 training and 2,621,440 validation samples), three paired seeds give mean AUC 0.657161 for HugeCTR and 0.657166 for RAPID. The mean paired difference (RAPID minus HugeCTR) is +0.000004 with a 95% confidence interval of [–0.000215, +0.000223]. The interval includes zero; it does not prove equivalence.

The additional 12 model/dataset pairs below use 510-step short training on 983,040 training samples and five paired seeds, not full-dataset convergence. Every reported paired-difference interval includes zero, but several are wide; the result is a limited accuracy check. LogLoss is computed from clipped predicted probabilities.

Table R8. Held-out AUC and LogLoss after 510 steps; paired AUC difference = RAPID minus HugeCTR.

Dataset / model	HugeCTR AUC	RAPID AUC	Mean paired difference [95% CI]	LogLoss, HugeCTR / RAPID
Kaggle / DLRM	0.626839	0.625624	–0.001216 [–0.010125, +0.007693]	2.320256 / 2.417471
Kaggle / DLRM-DCNv2	0.637947	0.640222	+0.002275 [–0.001915, +0.006466]	1.430427 / 1.357891
Kaggle / Wide&Deep	0.660461	0.659502	–0.000959 [–0.004375, +0.002456]	1.234775 / 1.233248
Kaggle / DeepFM	0.667539	0.667832	+0.000293 [–0.000472, +0.001059]	0.862612 / 0.862193
Terabyte / DLRM	0.589586	0.589612	+0.000026 [–0.008002, +0.008054]	0.493343 / 0.514816
Terabyte / DLRM-DCNv2	0.600504	0.601417	+0.000912 [–0.003169, +0.004993]	0.396073 / 0.393783
Terabyte / Wide&Deep	0.601201	0.600938	–0.000263 [–0.001930, +0.001405]	0.394008 / 0.401402
Terabyte / DeepFM	0.611441	0.611374	–0.000066 [–0.000578, +0.000446]	0.248894 / 0.249281
Taobao / DLRM	0.565991	0.566503	+0.000511 [–0.000519, +0.001542]	0.689493 / 0.689298
Taobao / DLRM-DCNv2	0.560156	0.560812	+0.000656 [–0.001369, +0.002681]	1.286928 / 1.267466
Taobao / Wide&Deep	0.568817	0.568714	–0.000103 [–0.000556, +0.000349]	1.250600 / 1.246514
Taobao / DeepFM	0.568164	0.568066	–0.000099 [–0.001140, +0.000943]	1.295433 / 1.281402

Proposed manuscript text — Section 5.1, after the MCR definition and Figure 6(c); Section 4.1, replace the accuracy-preservation qualification; Section 7.3, append the new accuracy results:

> We test whether high MCR is driven only by one- or two-occurrence IDs. In 2,097,152 Criteo multi-hot samples, we define target support as the number of distinct samples containing an ID and cold IDs as those seen in fewer than 0.01% of samples. Among cold targets with support of at least 10, 20, 50, and

100, the fractions having MCR>95% are 70.68%, 80.82%, 84.79%, and 85.50%. At the highest threshold, targets have appeared 100–209 times. This establishes the statistic for this generated multi-hot workload, where the expansion itself can induce co-occurrence; it does not establish the same proportion for unexpanded categorical data.

RAPID keeps the membership of each original global batch fixed and changes only its assignment to ranks. For rank-independent sample computation and identical optimizer semantics, the batch objective and aggregate gradient are unchanged mathematically; floating-point reduction order and rank-local state can produce implementation-level differences. We therefore check model accuracy empirically rather than claiming bitwise-identical training.

On a full one-epoch Taobao/DLRM run with 23,920,640 training and 2,621,440 validation samples, three paired seeds yield mean validation AUCs of 0.657161 for HugeCTR and 0.657166 for RAPID. The mean paired difference is +0.000004 with 95% interval [–0.000215, +0.000223]. A further five-seed, 510-step check across 12 dataset/model combinations finds that all paired AUC-difference intervals include zero; these short runs test model coverage and are not evidence of full convergence or statistical equivalence.

Meta Revision 6 — Correcting Figure 4 and Equation (5)'s counting convention

Response. Reviewer 5 is correct: the submitted Figure 4(b) Reduction curve counts only nonempty remote-owner contributions, which varies by table, while Equation (5) models a fixed-length aggregated output. We will correct Figures 4(b)–(c) and the all-Reduction curve of Figure 11 to a single forward remote-receive convention. At local batch B=8,192 and G=8 ranks, Reduction transfers B(G–1)=57,344 vectors per table per rank, or 1,490,944 for 26 tables. Equation (5)'s BG=65,536 per table counts the local contribution too; subtracting B gives actual remote payload. The old approximately 567,000-vector total in Figure 4(c) resulted from the nonempty-owner count and is not the fixed Reduction payload. HugeCTR's embedding communication implementation exchanges fixed output vectors in this aggregation path.

Counting scope, B=8,192 and G=8	Per table, per rank	26 tables
All aggregated slots, including local	65,536	1,703,936
Forward vectors received from other ranks	57,344	1,490,944

Proposed manuscript text — Section 3.2, replace Figure 4(b)–(c) and caption; Section 6.1, immediately after Equation (5); Section 7.4, replace the Figure 4 discussion; Figure 11, correct all-Reduction values:

> Equation (5) counts B·G aggregated output slots per table and receiving rank, including B locally produced slots. For transferred embedding payload we consistently count only vectors received from other ranks, B·(G–1). Thus, with eight ranks and local batch 8,192, fixed Reduction sends 57,344 remote vectors per table per rank, independent of table ID distribution or pooling factor; across 26 tables this is 1,490,944 forward remote vectors. Figure 4(b)–(c) and the all-Reduction series of Figure 11 use this same remote-only scope. The previous Figure 4 Reduction plot counted nonempty owner contributions instead of fixed aggregated output slots and has been replaced.

Response to Reviewer 2

R1, external comparison. Meta Revision 1 supplies seven-system measured throughput (Table R1), the T-series component comparison (Table R2), and exact proposed additions to Sections 7.1, 7.2, and 8. The differing HET-GMP optimizer and engine are stated with the result.

R2, physical scaling. Meta Revision 2 quotes replacement text for the abstract, Section 7.6, and conclusion that limits measured speedups to eight GPUs and calls 16–128 ranks trace-driven transfer studies.

R3, temporal robustness. Meta Revision 3 gives the four-strategy same-window comparison, zero existing-row migrations, and the precise limit that a shuffled single-day append stream does not measure temporal drift or planning wall time. It quotes replacement Section 7.8 text.

R4, batch size and peak load. Meta Revision 4 provides batch-dependent traffic, absolute and peak rank receives, capacity imbalance, and the proposed Section 5.2/7.8 wording. Throughput and gradient-accumulation sensitivity are not inferred from traffic counts.

R5, accuracy. Meta Revision 5 quotes the Section 4.1 and 7.3 changes, gives a full one-epoch three-seed Taobao comparison, and labels the 12 other combinations as five-seed short runs rather than full convergence.

Response to Reviewer 3

Physical scaling and cold-ID evidence. Meta Revisions 2 and 5 give replacement claims at the exact manuscript locations. The MCR table excludes low-support targets and explicitly scopes the result to generated multi-hot data.

Workload-aware alternatives. Meta Revision 1 adds HET-GMP to both the experiment and Section 8 related work, describing its graph-based co-occurrence partitioning and RAPID's distinct global-batch, ACP, capacity, and incremental-placement properties.

Parameters and order. Meta Revision 4 states the default β, γ, and Top-K, reports one-factor communication changes, defines reverse profiling order, and provides the proposed Section 5.2 text. No unmeasured optimality or METIS-seed sensitivity is claimed.

Response to Reviewer 5

Closest prior work and broader evaluation. Meta Revision 1 supplies the HET-GMP distinction and seven-system results, plus a Taobao workload and T-series shards. Meta Revision 2 makes the hardware limit explicit. The quoted Section 8 paragraph can be read directly here.

Figure 4 versus Equation (5). Meta Revision 6 supplies the corrected counting rule, the 57,344-per-table and 1,490,944-total values, and the exact Figure 4, Figure 11, and Section 6.1 replacement wording.

Research challenge. The proposed Section 8 text states what RAPID adds beyond graph-based co-placement: per-table strategy selection, preservation of each global batch, strict row-capacity balance, and placement of new IDs without migrating existing rows.